KIE3005 SEM1, 2023/2024 FINAL EXAM SOLUTIONS

No.	Answers	Marks
1. a. (i)	1. Minimization of Cost Function C(r): First, we need the derivative of the cost function C(r): $C'(r) = -1000/r^2 + 4\pi r$ The goal is to find the value of r that minimizes C(r), which occurs when C'(r) = 0. Using the Modified Secant Method: - First Iteration: Initial guess: $r_0 = 6$ $C'(6) = -1000/6^2 + 4\pi * 6 = -27.78 + 75.40 = 47.62$ Perturbation: $r_0 + \delta * r_0 = 6 + 0.01 * 6 = 6.06$ $C'(6.06) = -1000/6.06^2 + 4\pi * 6.06 = -27.24 + 76.14 = 48.90$ Applying the Modified Secant formula: $r_1 = 6 - 47.62 / ((48.90 - 47.62) / 0.06) = 6 - 47.62 / 21.33 = 6 - 2.23 = 3.77$ - Second Iteration: New guess: $r_1 = 3.77$ $C'(3.77) = -1000/3.77^2 + 4\pi * 3.77 = -70.37 + 47.33 = -23.04$ Perturbation: $r_1 + \delta * r_1 = 3.77 + 0.01 * 3.77 = 3.808$ $C'(3.808) = -1000/3.808^2 + 4\pi * 3.808 = -68.93 + 47.85 = -21.08$ Applying the formula again: $r_2 = 3.77 - (-23.04 / ((-21.08 - (-23.04)) / 0.038)) = 3.77 - (-23.04 / 51.58) = 3.77 + 0.447 = 4.217$	2 marks 1 mark 1 mark 1 mark 1 mark 1 mark 1 mark
(ii)	Why Use the Modified Secant Method compared traditional Secant Method: The Modified Secant Method is useful in this case because it **requires only a single initial guess r_0**, rather than two initial points, which may be difficult to choose in certain engineering problems. By introducing a small perturbation δ, we	1 mark

	can estimate the derivative in a way that still leads to convergence, even when information about the second point is unavailable. In this scenario, it allows us to efficiently approximate the radius that minimizes the cost function, while **avoiding potential oscillations or divergence** that can occur with the traditional secant method if the initial guesses are poor.	1 mark
(b) (i)	**Modified Newton-Raphson Method** Modified Newton-Raphson method to find the roots of the function f(x) = (x - 2)^2(x - 3), which has a multiple root at x = 2. **Function and Derivatives** Function: f(x) = (x - 2)^2(x - 3) First Derivative: f'(x) = (x - 2)(3x - 8) Second Derivative: f''(x) = 6x - 18 **Initial Values** Starting point: x0 = 2.5 **Iterations** **1st Iteration: x0 = 2.5** Calculate: f(2.5) = -0.125 f'(2.5) = -0.25 f''(2.5) = -3 Update: x1 = 2.5 - (-0.125 / (-0.25 - (-0.125 * -3 / (2 * -0.25))) = 2.75 **2nd Iteration: x1 = 2.75** Calculate: f(2.75) = -0.140625 f'(2.75) = 0.1875 f''(2.75) = 3 Update: x2 = 2.75 - (-0.140625 / (0.1875 - (-0.140625 * 3 / (2 * 0.1875))) x2 ≈ 2.857 **3rd Iteration: x2 ≈ 2.857** Calculate: f(2.857) ≈ -0.105 f'(2.857) ≈ 0.489 f''(2.857) ≈ 3.142 Update: x3 = 2.857 - (-0.105 / (0.489 - (-0.105 * 3.142 / (2 * 0.489))) x3 ≈ 2.986	2 marks 2 marks 2 marks

(ii)	**Summary** After three iterations, the estimated root of the equation f(x) = (x - 2)^2(x - 3) is approximately x ≈ 2.986. **Obtaining the Root at x = 3** 1. Initial Guess: - We started with an initial guess of x0 = 2.5. This choice led us to the nearby root at x = 3 rather than the double root at x = 2. 2. Iteration Process: - Through the iterations, we calculated f(x_n), f'(x_n), and f''(x_n) using the Modified Newton-Raphson formula. As we iterated, we found that our values were converging to approximately x = 2.986, which is indeed very close to the single root x = 3. **Why We Did Not Obtain the Other Root x = 2** 1. Choice of Initial Guess: - Although x0 = 2.5 is numerically closer to x = 2, the behavior of the function and its derivatives influenced the convergence towards x = 3. 2. Root Characteristics: - The function has a double root at x = 2, which means that it flattens out at this point. Hence, the Modified Newton-Raphson method can struggle to converge to x = 2 if starting close to it, especially given that x = 3 is also a valid root.	2 marks 2 marks
2 (a) (i) (ii)	RK4 method as it's uses the highest order compared to the other 3 methods employed. Heun method is 2nd order method whereas RK4 is fourth order method. The Euler and Implicit Euler are 1st order methods. Since higher order methods gives less error that 1st orders the results of using Heun and RK4 are closer to each other. function [ts, ys]=ModEuler(dydt,tv,y0,N)	1 mark 3 marks

(iii)

```
ts =tv(1);                    % initial t
y =y0    ;                    % initial y
tstep = (tv(2)-tv(1))/N;            % time step
                 % number of steps to go
oldy = y(:,end);
ys(1,:)=oldy ;
for i=1:N
   oldt = ts(i);          % pick the last entry of array
   newt = oldt + tstep;         % step forward in t
   newy = fsolve(@(y) y - oldy- tstep*dydt(newt,y), oldy);  % backward Euler
   newy=oldy+tstep*dydt(newt,0.5*(newy+oldy));
   oldy=newy;
   ts(i+1) = newt;
   ys(i+1,:) = newy;   % store y(1),y(2) in row of array ys
end
```

1 mark

3 marks

2 marks

(b)

$$\frac{v_s}{R_1CL} = \frac{d^2i(t)}{dt^2} + \left(\frac{R_2}{L} + \frac{1}{R_1C}\right)\frac{di(t)}{dt} + \left(\frac{R_1+R_2}{R_1CL}\right)i(t)$$

(i)

We reduce the second order form of the equation into first order

Lets set ,

$$\frac{di(t)}{dt} = a(t) \quad i(0) = 0$$

1 mark

Hence,

$$\frac{da(t)}{dt} = \frac{v_s}{R_1CL} - \left(\frac{R_2}{L} + \frac{1}{R_1C}\right)a(t) - \left(\frac{R_1+R_2}{R_1CL}\right)i(t) \quad a(0) = unknown$$

2 marks

Shooting method was implemented to solve this question as a(0) is to be predicted. For shooting method shown below, Euler's method was combined with false position root finding approach. The answer can be found via other ode solver combined with other root finding methods. However Newton Rapshson will fail though as it requires differentiation of equation that will cause it to vanish.

```
clear all
```

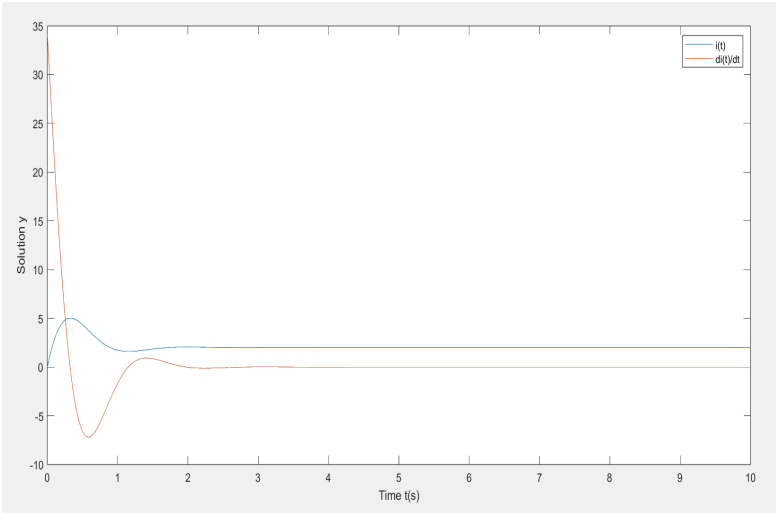
(ii)	tic
	fxr=100;
	Ea=100;
	xr=5;
	xl=0;
	xu=50;
	Vs=20;
	R1=5;
	R2=5;
	C=0.1;
	L=1;
	f1= @(t,y) [y(2); (Vs/(R1*C*L))-((R2/L)+(1/R1*C))*y(2)-(R1+R2)/(R1*C*L)*y(1)];% define function f(t,y)
	% initial condition with vector y0
	init1=0;
	t0 = 0;T=10;
	tv=[t0 T];
	Nx=1000;
	
	yinit = [init1;xl];
	[ts,ys] = Euler(f1,tv,yinit,Nx);
	y5=ys(end,1);% based on initial value of xl
	fxl=y5-2;
	yinit = [init1;xu];
	[ts,ys] = Euler(f1,tv,yinit,Nx);
	y5=ys(end,1);% based on initial value of xu
	fxu=y5-2;
	i=0;
	%%%false position method%%%%%%%%%%%%%%%%%%
	while abs(Ea)>0.001
	  xold=xr; %storing previous xr
	  xr=(-fxl*xu+fxu*xl)/(fxu-fxl);
	
	yinit = [init1;xr];
	[ts,ys] = Euler(f1,tv,yinit,Nx);
	y5=ys(end,1);% based on initial value of xr
	fxr=y5-2;
	
	  if fxr*fxl>0
	  xl=xr;
	  fxl=fxr;
	  else
	  xu=xr;
		2 marks
		
		2 marks

```
    fxu=fxr;
 end

 i=i+1;
 Ea=100*abs((xr-xold)/xr);
end
toc
yinit = [init1;xr];
[ts,ys] = Euler(f1,tv,yinit,Nx);
y5=ys(end,1)
i
Ea
xr
figure(1)
plot(ts,ys);
legend('Euler');
xlabel('Time t');
ylabel('Solution y');
```

2 marks

1 mark



(viewing only)-no marks